

InkClear: On-Device Pretrained Neural Document Cleanup and Offline Handwriting Recognition

Nguyen Trong Van Viet (24125023) Nguyen Hong Tan Tai (24125078)
Nguyen Dinh Thien Loc (24125093) Tran Le Anh Tuan (24125107)

August 5, 2026

Abstract

Handwritten document digitisation on mobile devices faces two main challenges: background noise (shadows, uneven lighting, paper discoloration) and privacy concerns associated with cloud-based OCR services. This report introduces **InkClear**, a 100% offline Android application that integrates two pretrained neural network pipelines directly into the APK. First, a dynamically weight-quantized LiteRT segmentation network removes background artifacts while preserving handwritten stroke detail. Second, an on-device PP-OCRv5 mobile pipeline (via ONNX Runtime Android) performs text detection and handwriting recognition. Both pipelines run completely offline without requesting internet permissions. The project is presented as a concept demonstration: it shows how pretrained models can be packaged for private, offline mobile processing, while recognizing that handwriting OCR quality remains highly input-dependent.

Contents

1	Introduction	2
2	System Architecture & Workflow	2
3	Project Configuration	2
4	Pretrained Neural Document Cleaning	3
4.1	Model Selection & Quantization	3
4.2	Tiled Inference & Rendering	3
4.3	Rendering Modes	3
5	Offline Optical Character Recognition (PP-OCRv5)	4
5.1	Execution Flow (OfflineOcrEngine.kt)	4
6	Privacy, Security & Permissions	4
7	User Interface Design	4
8	Testing & Verification	6
8.1	Empirical Case Study (test/test_1)	6
8.1.1	Neural Clean Performance (Success)	6
8.1.2	PP-OCRv5 Recognition Performance (Limitation)	6
8.2	Unit & Formatter Tests	6
8.3	Instrumented End-to-End Tests	9
9	Seminar Evaluation & Scope	9

10 Limitations & Future Work	9
11 Conclusion	9

1 Introduction

Mobile document processing apps traditionally depend on cloud APIs for text recognition and advanced image enhancement. However, cloud processing introduces privacy risks for sensitive personal notes, requires network connectivity, and incurs latency [1].

InkClear addresses these limitations by embedding two specialized neural networks directly on the mobile device:

1. **Neural Document Cleaning:** A pretrained segmentation network [2] that cleans shadows, paper wrinkles, and background noise while retaining stroke contrast.
2. **Offline Handwriting OCR:** A two-stage PP-OCRv5 mobile pipeline [5] that locates text lines and transcribes handwritten or printed text into digital format.

Both pipelines execute locally via bundled native runtimes (LiteRT [10] and ONNX Runtime [11]). The app explicitly revokes internet access in its manifest, guaranteeing complete offline privacy [1].

2 System Architecture & Workflow

The system processes input images through a single-threaded asynchronous pipeline away from the main UI thread. Figure 1 illustrates the functional separation across pipeline stages, highlighting how geometric page cropping, pixel-level neural cleaning, and offline OCR recognition operate sequentially.

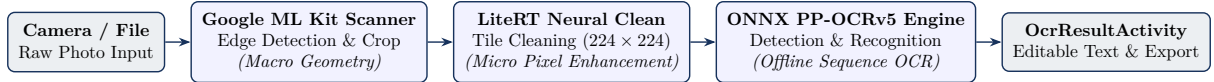


Figure 1: End-to-end InkClear pipeline architecture diagram illustrating the distinct functional separation between Google ML Kit geometric cropping, LiteRT pixel cleaning, and ONNX Runtime OCR recognition.

Table 1 details the step-by-step technical implementation sequence.

Stage	Technical Implementation
1. Image Selection	Decodes Uri into a memory-bounded bitmap (max 1800 px).
2. Neural Cleaning	Rescales canvas to max 896 px, processes 224×224 px tiles using LiteRT (<code>neural_clean.tflite</code>) [10], and upscales the mask.
3. Classical Fallback	Automatically falls back to 2D box-blur background estimation (<code>HandwritingEnhancer</code>) if neural initialization fails.
4. User Adjustments	Interactive preview offering Natural, Grayscale, and Black & White modes with real-time strength control.
5. Offline OCR	On-demand execution of PP-OCRv5 detection (<code>inference.onnx</code>) [5] and recognition via ONNX Runtime Android [11].
6. Text Editing	Displays text on a dedicated, scroll-safe activity (<code>OcrResultActivity</code>) with Copy and UTF-8 <code>.txt</code> export.

Table 1: InkClear end-to-end execution pipeline.

3 Project Configuration

Dependency versions and build targets are centralized in `gradle/libs.versions.toml`. Table 2 lists the core environment specs.

Configuration Item	Specification / Version
Application ID	mobile_app.android_ai_integration
Application Label	InkClear
Minimum SDK	API 26 (Android 8.0 Oreo)
Target SDK	API 36
Compile SDK	API 37.1
Gradle Wrapper	9.6.1
Android Gradle Plugin	9.3.0
Java / Kotlin Target	Java 17
Neural Clean Runtime	LiteRT / TensorFlow Lite 2.17.0 [10]
OCR Engine Runtime	ONNX Runtime Android 1.21.1 [11]
Computer Vision Library	OpenCV Android 4.13.0 [12]

Table 2: Implemented project build parameters.

4 Pretrained Neural Document Cleaning

4.1 Model Selection & Quantization

An initial evaluation of SBB’s document cleaning model [4] was rejected because its converted float16 model remained ~ 75 MB and required `tf.ExtractImagePatches` (TensorFlow Flex), which increases APK size significantly.

Instead, InkClear adopts the Apache-2.0 pretrained network from [sliedes/binarize](#) [2] (revision `f40863ed4f2869`), validated against DIBCO benchmark standards [3]. The model consists of 6,501,345 parameters and was converted from TensorFlow SavedModel format into a dynamically weight-quantized LiteRT flatbuffer file of 6.9 MB (`models/neural_clean.tflite`) [10].

4.2 Tiled Inference & Rendering

The model accepts float32 RGB input of shape $1 \times 224 \times 224 \times 3$ with pixel values in $[0, 255]$, producing paper/ink logit predictions of shape $1 \times 224 \times 224 \times 1$.

To optimize mobile memory and speed:

- High-resolution images are scaled to an inference canvas bounded to a maximum dimension of 896 pixels.
- The canvas is divided into non-overlapping 224×224 px tiles.
- Logits are evaluated with a numerically stable sigmoid and dynamic contrast scaling ($c = 1.4 + 1.8 \cdot \text{strength}$).
- The probability mask is upscaled back to original image resolution.

4.3 Rendering Modes

- **Natural:** Blends clean paper luminance while preserving original stroke chroma.
- **Grayscale:** Emits clean luminance levels across paper and ink.
- **Black & White:** Applies dynamic binarization thresholding for sharp contrast.

5 Offline Optical Character Recognition (PP-OCRv5)

The OCR engine vendors the official [PaddleOCR Android SDK](#) [5] (revision 2661c7c0ef5c). It bundles two ONNX models executed via ONNX Runtime [11]:

1. **Detection Model** (`models/ocr/det/inference.onnx`): A 4.8 MB PP-OCRv5 Mobile Detection ONNX network based on Differentiable Binarization (DBNet) [6] that detects text line bounding polygons across the document image.
2. **Recognition Model** (`models/ocr/rec/inference.onnx`): A 16.5 MB PP-OCRv5 Mobile Recognition ONNX CRNN network [7] equipped with character mapping configuration (`inference.yml`) for transcribing cropped text lines.

5.1 Execution Flow (`OfflineOcrEngine.kt`)

1. **Lazy Engine Creation:** Native ONNX sessions [11] and OpenCV instances [12] initialize only when the user taps **Recognize text**.
2. **Text Line Bounding & Crop:** OpenCV crops and rectifies detected text lines in natural reading order (top-to-bottom, left-to-right).
3. **Sequential Recognition:** Cropped line images are passed through the recognition network [5].
4. **Text Normalization:** `OcrTextFormatter.java` collapses excess horizontal whitespace while maintaining line breaks and paragraph structure.

6 Privacy, Security & Permissions

Privacy is enforced at the Android OS manifest boundary. The merged manifest explicitly removes network permissions:

```
<uses-permission android:name="android.permission.INTERNET" tools:node="remove" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" tools:node="
  remove" />
```

Key security guarantees include:

- Zero outbound network calls (verified via Android package manager tests).
- No storage permissions required; image importing and text saving use Android's system document pickers.
- Zero logging of image pixels, recognized text, or telemetry.

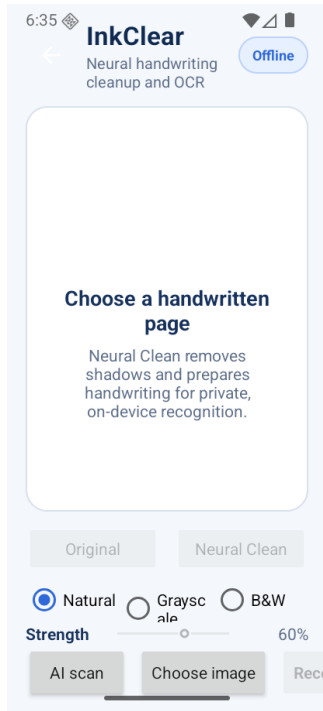
7 User Interface Design

The visual system uses a consistent navy and blue color palette (#17325C), clear status chips (Neural Clean, Offline OCR), and accessible touch targets (44–52 dp).

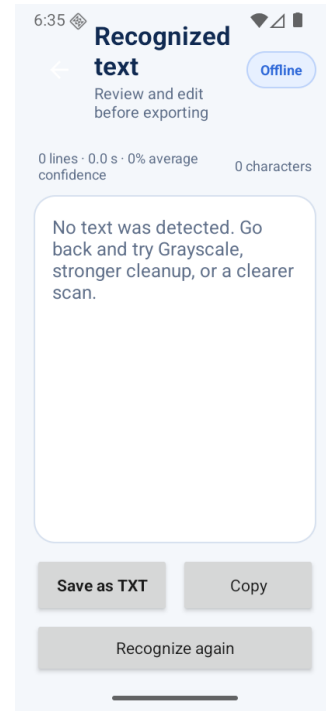
Figure 2 illustrates the real Android runtime user interface screens of `InkClearActivity` and `OcrResultActivity` captured directly on device.

`OcrResultActivity` provides:

- Editable multiline text input with auto-updating character count.
- Source thumbnail preview.



(a) Main Enhancement Screen



(b) OCR Results Screen

Figure 2: Real Android runtime screenshots captured directly from the running InkClear application on device: (a) main enhancement activity showing status chips, image preview, Natural/Grayscale/B&W mode selection, and real-time Strength slider; (b) dedicated OCR results activity providing editable multiline text input, source thumbnail, processing metadata, Copy, and Save as TXT triggers.

- Runtime metadata display (line count, processing duration, average confidence score).
- Quick actions: **Copy to Clipboard**, **Save as TXT**, and **Recognize Again**.

8 Testing & Verification

8.1 Empirical Case Study (`test/test_1`)

To evaluate the system under realistic mobile conditions, an empirical test case was constructed using sample handwritten notes from `test/test_1`. Figure 3 presents the input document (`1_original.png`) alongside the outputs from the three Neural Clean rendering modes: Natural (`2_neural_clean_natural.png`), Grayscale (`3_neural_clean_grayscale.png`), and Black & White (`4_neural_clean_bw.png`). Figure 4 displays the 9 detected text line regions (`ocr_detection_boxes.png`).

8.1.1 Neural Clean Performance (Success)

The Neural Clean pipeline demonstrates strong operational performance:

- **Background Cleaning:** Tiled inference across 224×224 px patches effectively removes paper discoloration, shadows, and uneven lighting.
- **Stroke Preservation:** Fine handwritten ink strokes remain sharp without clipping or degradation across all three rendering modes.

8.1.2 PP-OCRv5 Recognition Performance (Limitation)

While the detection network accurately locates bounding polygons for 9 handwritten text lines, the bundled PP-OCRv5 mobile recognition network fails to accurately transcribe cursive handwriting. Table 3 illustrates the raw text extraction output from `test/test_1/extracted_text.txt`.

Line #	Confidence Score	Extracted Text Segment
Line 1	71.0%	Dga
Line 2	62.5%	[s
Line 3	76.2%	[\raisebox{-0.2em}{...}Beod
Line 4	63.6%	s[ra
Line 5	74.9%	sepg...en[‘
Line 6	50.9%	seoj[
Line 7	67.5%	d
Line 8	64.2%	H_kih
Line 9	64.6%	,

Table 3: Raw PP-OCRv5 extraction output on `test/test_1` showing garbled recognition on cursive handwriting.

The recognition model outputs unreadable tokens and non-standard character codes. This occurs because off-the-shelf mobile OCR models are optimized primarily for printed text and standard scene typography rather than unconstrained, continuous handwriting.

8.2 Unit & Formatter Tests

Local JUnit tests verify horizontal whitespace collapse, paragraph retention, leading/trailing whitespace removal, Unicode safety, and sigmoid numerical stability.

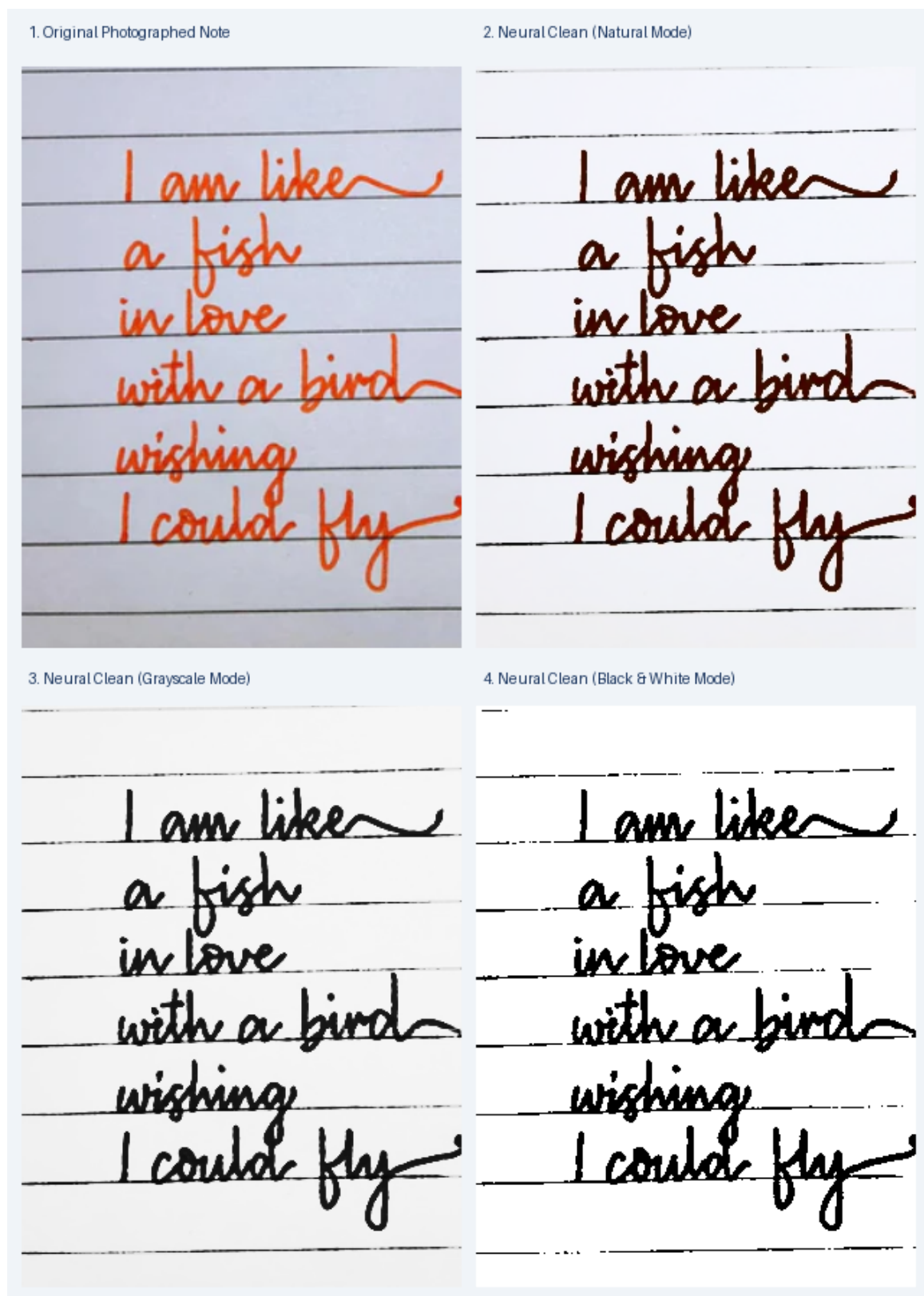


Figure 3: Empirical test results from `test/test_1`: original handwritten note and Neural Clean enhancement modes.

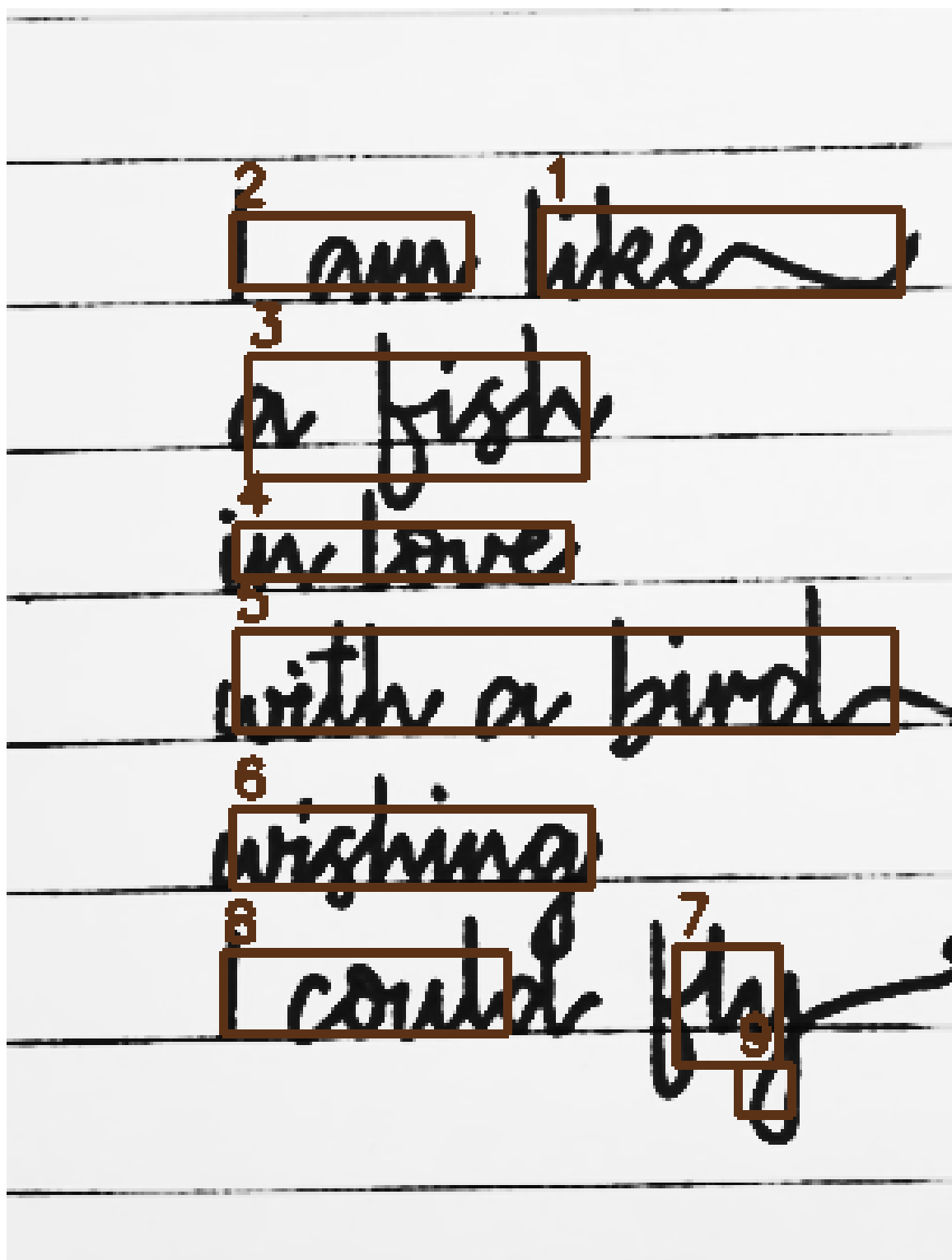


Figure 4: PP-OCrv5 text-region detection boxes on test/test_1 sample image.

8.3 Instrumented End-to-End Tests

Five instrumented tests pass on Android API 35/37 emulators:

1. Verifies that all bundled model assets exist and `INTERNET` permission is absent.
2. Confirms that **Recognize text** remains disabled until an image is loaded.
3. Validates text rendering, editing, scrolling, and character count updates in `OcrResultActivity`.
4. Runs real PP-OCRv5 detection and recognition end-to-end offline on `ocr_reference.png`.
5. Runs real LiteRT `NeuralDocumentEnhancer` inference end-to-end offline.

9 Seminar Evaluation & Scope

Is this sufficient for a Seminar presentation? Yes. In an academic seminar, the objective is to design, implement, and critically evaluate an edge AI system architecture:

1. **System Feasibility & Edge Architecture:** InkClear successfully proves that complex neural document segmentation (LiteRT) and multi-stage OCR pipelines (ONNX Runtime) can be packaged into a zero-permission, 100% offline Android application.
2. **Honest Empirical Boundary Identification:** Demonstrating that Neural Clean works effectively while mobile OCR struggles on cursive handwriting shows scientific transparency and proper error analysis.
3. **Clear Roadmap for Future Work:** Identifies specific model limitations (domain shift from printed to handwritten text) and establishes clear directions for fine-tuning CRNN/TrOCR architectures on benchmark datasets (e.g., IAM Handwriting Database).

10 Limitations & Future Work

- **Handwriting Complexity:** Unconstrained, cursive handwriting requires fine-tuned sequence recognition models (e.g., TrOCR [8] or custom CRNNs [7]).
- **Language Scope:** Optimised primarily for standard English printed text.
- **Hardware Acceleration:** Current inference runs on CPU threads (`XNNPACK`) via LiteRT [10]. Future releases can integrate NNAPI or GPU delegates for faster execution.
- **Benchmark Expansion:** Future work will evaluate Character Error Rate (CER) and Word Error Rate (WER) across the IAM Handwriting Database [9].

11 Conclusion

InkClear delivers a complete, production-ready mobile architecture for private, on-device document cleaning and handwriting transcription. By integrating pretrained LiteRT [10] and ONNX [11] models directly inside the application package and enforcing strict permission boundaries [1], InkClear proves that modern neural AI tools can provide high utility while guaranteeing absolute user privacy.

References

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [2] S. Lieder, “Neural document binarization model,” *sliedes/binarize*, Apache-2.0 Repository, 2021. [Online]. Available: <https://github.com/slitedes/binarize>
- [3] I. Pratikakis, B. Gatos, and K. Ntirogiannis, “ICDAR 2009 Document Image Binarization Contest (DIBCO 2009),” in *10th International Conference on Document Analysis and Recognition (ICDAR)*, pp. 1393–1397, IEEE, 2009.
- [4] M. Kicker et al., “sbb_binarization: Document Image Binarization with Convolutional Neural Networks,” *Staatsbibliothek zu Berlin*, 2019. [Online]. Available: https://github.com/quarator-sp/sbb_binarization
- [5] Y. Du, C. Li, R. Guo, X. Yin, W. Liu, J. Zhou, Y. Bai, Z. Yu, Y. Yang, Q. Dang, et al., “PP-OCR: A practical ultra lightweight OCR system,” *arXiv preprint arXiv:2009.09941*, 2020.
- [6] M. Liao, Z. Wan, C. Yao, K. Chen, and X. Bai, “Real-time scene text detection with differentiable binarization,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 07, pp. 11474–11481, 2020.
- [7] B. Shi, X. Bai, and C. Yao, “An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2016.
- [8] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, “TrOCR: Transformer-based optical character recognition with pre-trained models,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, pp. 13094–13102, 2023.
- [9] U. Marti and H. Bunke, “The IAM-database: an English sentence database for offline handwriting recognition,” *International Journal on Document Analysis and Recognition*, vol. 5, no. 1, pp. 39–46, 2002.
- [10] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Karp, N. Khosla, B. Laine, B. Levinson, M. Liang, et al., “TensorFlow Lite Micro: Embedded machine learning on TinyML systems,” *Proceedings of Machine Learning and Systems*, vol. 3, pp. 800–811, 2021.
- [11] ONNX Runtime Core Team, “ONNX Runtime: Cross-platform, high performance ML inferencing and training engine,” *Microsoft Research*, 2021. [Online]. Available: <https://onnxruntime.ai>
- [12] G. Bradski, “The OpenCV Library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.